UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
SI806 · Desarrollo Adaptativo e Integrado del Software

TALLER 3 · DESARROLLO E INTEGRACIÓN DE UNA API REST

Documentación de la API REST

Luxxito Sports: endpoints, contratos JSON, seguridad y códigos HTTP

Referencia de la API REST propia del proyecto: recursos, reglas de negocio que aplica el servidor, autenticación con JWT y roles, titularidad de reservas, concurrencia y ejemplos de solicitudes y respuestas.

Integrantes

Steven Cadillo
Cliente móvil, pagos, pruebas y demostración

Elvis Arboleda
Base de datos Supabase y despliegue

Juan Medina
Backend, Render e infraestructura

Repositorio en GitHub (rama main)
`https://github.com/Arbolito-ctrl/LuxxitoSports/tree/main`

API pública
`https://luxxito-sports-api.onrender.com/api`

Lima, noviembre de 2026

Índice

Datos generales 2

1. Resumen de Endpoints Propios 2

2. Seguridad JWT y Control de Roles (LX-201) 3

2.1. Titularidad de reservas 3

2.2. Contraseñas 4

2.3. Concurrencia y unicidad 4

3. Detalle de Endpoints y Ejemplos JSON 4

3.1. GET /api/canchas/:id/disponibilidad?fecha=2026-11-10&hora=18 4

3.2. POST /api/reservas 5

3.3. POST /api/reservas/:id/pagos y PATCH /api/pagos/:id/aprobar 6

3.4. GET /api/pagos?estado=PENDIENTE_REVISION y PATCH /api/pagos/:id/rechazar (Rol… 6

3.5. POST /api/pagos/comprobante (foto del voucher) 6

3.6. POST /api/reservas/:id/cancelar 7

Datos generales

Tabla 1. *Datos generales de la API.*

Aspecto	Detalle
URL Base (Producción, Render)	`https://luxxito-sports-api.onrender.com/api`
URL Base (Desarrollo Local)	`http://localhost:3000/api`
URL Base (Emulador Android)	`http://10.0.2.2:3000/api`
Formato de Intercambio	`application/json; charset=utf-8`
Autenticación y Autorización	JWT (Authorization: Bearer <token>) firmado con JWT_SECRET, vigente 30 días (exp). Roles JUGADOR y ADMIN.
Contraseñas	se guardan con hash scrypt y sal aleatoria por usuario (scrypt$<sal>$<hash>); nunca en texto plano.
Contrato	contracts/openapi.yaml (versión 1.6.0) · historial en contracts/CHANGELOG.md.
Persistencia	PostgreSQL en Supabase (usuarios, complejos, canchas, extras, reservas, reserva_extras, pagos) con Row Level Security (RLS) y bucket privado comprobantes.

Este documento se genera a partir de `docs/API_DOCUMENTATION.md`. El contrato formal está en `contracts/openapi.yaml` (OpenAPI 3.0.3) y puede abrirse en Swagger Editor o Redocly.

1. Resumen de Endpoints Propios

#	Método	Ruta	Operación / Regla de Negocio	Códigos HTTP
1	`POST`	`/api/auth/login`	Autenticación de usuario (JUGADOR o ADMIN) y emisión de JWT	200, 400, 401
2	`POST`	`/api/auth/registro`	Registro de cuenta nueva (JUGADOR) con validación de DNI/email y emisión de JWT	201, 400, 409
3	`GET`	`/api/health`	Estado de la API, versión (api_version), commit desplegado y modo de conexión a Supabase	200, 503
3b	`GET`	`/api/version?canal=latest-apk`	Última versión publicada del APK (lee version.json de GitHub Releases)	200, 502
4	`GET`	`/api/complejos`	Listar complejos deportivos (filtro por distrito)	`200`
5	`GET`	`/api/complejos/:id`	Detalle de complejo con sus canchas y extras	200, 400, 404
6	`GET`	`/api/canchas`	Listar canchas con filtros (complejo_id, distrito, deporte, estado)	`200`
7	`GET`	`/api/canchas/:id`	Obtener detalle de una cancha específica	200, 400, 404
8	`POST`	`/api/canchas`	CRUD (Create, Rol ADMIN): Registrar nueva cancha con validación de tarifas y nombre único por complejo	201, 400, 401, 403, 404, 409
9	`PUT`	`/api/canchas/:id`	CRUD (Update, Rol ADMIN): Actualizar tarifas, estado o características de cancha	200, 400, 401, 403, 404, 409
9b	`POST`	`/api/canchas/:id/imagen`	Foto de la cancha (Rol ADMIN): sube la imagen (JPEG/PNG/WEBP, base64) al bucket público canchas y guarda imagen_url	200, 400, 401, 403, 404, 413, 502
10	`GET`	`/api/canchas/:id/disponibilidad`	Lógica de Negocio 1: Grilla horaria por fecha, tarifas diurna/nocturna y stock de extras	200, 400, 404
11	`POST`	`/api/reservas/cotizar`	Lógica de Negocio 2: Cotización en servidor (cancha + extras + seña 50%) y verificación de stock	200, 400, 404, 409, 422
12	`GET`	`/api/reservas`	CRUD (Read, con sesión): Listar reservas con extras y pagos (estado, cliente_dni, fecha). El JUGADOR solo ve las suyas	200, 401

#	Método	Ruta	Operación / Regla de Negocio	Códigos HTTP
13	`GET`	`/api/reservas/:id`	Detalle completo de una reserva (solo titular o ADMIN)	200, 400, 401, 403, 404
14	`POST`	`/api/reservas`	CRUD (Create) + Lógica de Negocio 3: Crear reserva con bloqueo de 20 min, exclusión mutua de horario (también ante pedidos simultáneos) y reserva de extras	201, 400, 404, 409, 422
15	`PUT`	`/api/reservas/:id`	CRUD (Update): Actualizar datos de contacto o notas de una reserva activa (solo titular o ADMIN)	200, 400, 401, 403, 404, 409
16	`DELETE`	`/api/reservas/:id`	CRUD (Delete): Eliminar físicamente una reserva no confirmada (solo titular o ADMIN)	200, 400, 401, 403, 404, 409
17	`POST`	`/api/reservas/:id/pagos`	Lógica de Negocio 4: Registrar comprobante Yape/Plin/Transferencia y proteger reserva en revisión (solo titular o ADMIN)	201, 400, 401, 403, 404, 409, 422
18	`PATCH`	`/api/pagos/:id/aprobar`	Lógica de Negocio 5 (Rol ADMIN): Aprobación idempotente de pago y emisión de Ticket QR (LUXXITO-QR-...)	200, 400, 401, 403, 404, 409
19	`GET`	`/api/pagos?estado=PENDIENTE_REVISION`	Bandeja del operador (Rol ADMIN): Listar pagos con el resumen de su reserva	200, 400, 401, 403
20	`PATCH`	`/api/pagos/:id/rechazar`	Rechazo de comprobante (Rol ADMIN): observacion obligatoria; la reserva vuelve a PENDIENTE_PAGO con nuevo bloqueo de 20 min (idempotente)	200, 400, 401, 403, 404, 409
22	`POST`	`/api/pagos/comprobante`	Foto del voucher: sube la imagen (JPEG/PNG/WEBP, máx. 5 MB, en base64) al bucket privado comprobantes y devuelve comprobante_url	201, 400, 404, 409, 413, 422, 502
23	`POST`	`/api/reservas/verificar-qr`	Ingreso en puerta (Rol ADMIN): valida el Ticket QR una sola vez	200, 400, 401, 403, 404, 409
21	`POST`	`/api/reservas/:id/cancelar`	Lógica de Negocio 6: Cancelación con política de 24 horas (100% reembolso >= 24h; 50% retención < 24h) y revocación de QR (solo titular o ADMIN)	200, 400, 401, 403, 404, 409

2. Seguridad JWT y Control de Roles (LX-201)

Los endpoints del **Modo Operador** (`GET /api/pagos`, `PATCH /api/pagos/:id/aprobar`, `PATCH /api/pagos/:id/rechazar` y `POST /api/reservas/verificar-qr`) requieren el encabezado:

```
Authorization: Bearer <token_jwt>
```

- **Sin token o con firma JWT inválida:** responde `401 Unauthorized` (`code: "UNAUTHORIZED"`).
- **Con token válido de rol JUGADOR:** responde `403 Forbidden` (`code: "FORBIDDEN"`).
- **Con token válido de rol ADMIN:** permite listar, aprobar y rechazar comprobantes, y registrar ingresos con el Ticket QR.
- **Token vencido** (`exp`, 30 días desde el login): responde `401` y la app pide iniciar sesión de nuevo.

El catálogo (`POST /api/canchas`, `PUT /api/canchas/:id`, `POST /api/canchas/:id/imagen`) también es solo para `ADMIN`.

2.1. Titularidad de reservas

`GET /api/reservas` exige sesión. `GET`, `PUT` y `DELETE /api/reservas/:id`, `POST /api/reservas/:id/pagos` y `POST /api/reservas/:id/cancelar` solo los puede usar el **titular** (el DNI del token coincide con `cliente_dni`) o un `ADMIN`:

- Sin token: `401 UNAUTHORIZED`.
- Reserva de otra persona: `403 FORBIDDEN`.
- En el listado, un JUGADOR solo recibe sus reservas aunque envíe el `cliente_dni` de otra persona.

2.2. Contraseñas

`POST /api/auth/registro` guarda la contraseña con `scrypt` (sal de 16 bytes, hash de 64 bytes) y `POST /api/auth/login` la compara en tiempo constante. Si una cuenta antigua aún tenía la contraseña en texto plano, se reemplaza por su hash en el primer inicio de sesión correcto.

2.3. Concurrencia y unicidad

- Dos pedidos simultáneos por el mismo turno: solo uno crea la reserva y el otro recibe `409 CONFLICT` ("El turno elegido acaba de ser reservado por otra persona"). Lo garantiza la restricción de exclusión `reservas_sin_solapamiento` en PostgreSQL, y una cola por fecha en el servidor ordena los pedidos.
- Aprobar, rechazar, cancelar o validar el QR de la misma reserva a la vez se procesa en orden: el QR se acepta una sola vez y la segunda operación responde `409`.
- El nombre de una cancha es único dentro de su complejo (sin distinguir mayúsculas): un duplicado responde `409`.

3. Detalle de Endpoints y Ejemplos JSON

3.1. GET /api/canchas/:id/disponibilidad?fecha=2026-11-10&hora=18

Calcula para cada turno de 1 hora si está `DISPONIBLE`, `BLOQUEADA`, `OCUPADA`, `MANTENIMIENTO` o `PASADA`, asigna tarifa `DIURNA` (< 18:00) o `NOCTURNA` (>= 18:00) y calcula el stock restante de cada extra en el intervalo.

Respuesta 200 OK:


```
{
  "ok": true,
  "data": {
    "cancha": {
      "id": 1,
      "complejo_id": 1,
      "complejo_nombre": "Luxxito Arena San Miguel",
      "distrito": "San Miguel",
      "nombre": "Cancha Monumental 1",
      "deporte": "Futbol 7",
      "precio_hora_diurno": 90,
      "precio_hora_nocturno": 120,
      "estado": "ACTIVA"
    },
    "fecha": "2026-11-10",
    "horario_operacion": { "apertura": 7, "cierre": 23 },
    "turnos": [
      {
        "hora_inicio": 17,
        "hora_fin": 18,
        "etiqueta": "17:00 - 18:00",
        "tarifa_tipo": "DIURNA",
        "precio": 90,
        "estado": "DISPONIBLE"
      },
      {
        "hora_inicio": 18,
        "hora_fin": 19,
        "etiqueta": "18:00 - 19:00",
        "tarifa_tipo": "NOCTURNA",
        "precio": 120,
        "estado": "OCUPADA",
```

```
        "reserva_id": 4
      }
    ],
    "extras_disponibles": [
      {
        "id": 1,
        "complejo_id": 1,
        "nombre": "Juego de Chalecos (x12)",
        "precio_unitario": 15,
        "stock_total": 4,
        "stock_disponible": 4
      }
    ]
  }
}
```

3.2. POST /api/reservas

Crea una reserva verificando horario de atención, no solapamiento con otra reserva activa, stock de extras en el intervalo y cotizando en el servidor.

Cuerpo de Solicitud (Request Body):

```
{
  "cancha_id": 1,
  "cliente_nombre": "Carlos Mendoza",
  "cliente_dni": "74125896",
  "cliente_telefono": "999888777",
  "cliente_email": "carlos.mendoza@uni.pe",
  "fecha": "2026-11-10",
  "hora_inicio": 19,
  "hora_fin": 21,
  "notas": "Torneo interfacultades UNI",
  "extras": [
    { "extra_id": 1, "cantidad": 2 },
    { "extra_id": 2, "cantidad": 1 }
  ]
}
```

Respuesta 201 Created:

```
{
  "ok": true,
  "message": "Reserva creada con bloqueo temporal de 20 minutos.",
  "data": {
    "id": 1,
    "cancha_id": 1,
    "cancha_nombre": "Cancha Monumental 1",
    "complejo_nombre": "Luxxito Arena San Miguel",
    "cliente_nombre": "Carlos Mendoza",
    "cliente_dni": "74125896",
    "fecha": "2026-11-10",
    "hora_inicio": 19,
    "hora_fin": 21,
    "subtotal_cancha": 240,
    "subtotal_extras": 40,
    "total": 280,
    "monto_pagado": 0,
    "monto_devolucion": 0,
    "estado": "PENDIENTE_PAGO",
    "codigo_qr": null,
    "expires_at": "2026-11-03T05:05:00.000Z"
  }
}
```

3.3. POST /api/reservas/:id/pagos y PATCH /api/pagos/:id/aprobar

Registra el comprobante de pago (`YAPE`, `PLIN` o `TRANSFERENCIA`) bajo modalidad `SENA_50` (50%) o `TOTAL` (100%).

Cuerpo de Solicitud (POST /api/reservas/1/pagos):

```
{
  "metodo": "YAPE",
  "modalidad": "TOTAL",
  "monto": 280,
  "numero_operacion": "YAPE-9948102",
  "auto_aprobar": true
}
```

3.4. GET /api/pagos?estado=PENDIENTE_REVISION y PATCH /api/pagos/:id/rechazar (Rol ADMIN)

Bandeja de revisión del operador. Requiere `Authorization: Bearer <token_admin>`. Cada pago incluye un resumen de su reserva (cancha, jugador, fecha y horario) para revisarlo sin consultas adicionales. Un `estado` fuera de `PENDIENTE_REVISION`, `APROBADO`, `RECHAZADO` o `REEMBOLSADO` responde `400`.

Cuerpo de Solicitud (PATCH /api/pagos/3/rechazar):

```
{ "observacion": "Número de operación no coincide con el voucher" }
```

Reglas:

- Requiere token JWT válido con `rol: 'ADMIN'` (`401` sin token, `403` si es `JUGADOR`).
- `observacion` es obligatoria (mínimo 5 caracteres) » `400` si falta.
- Solo se rechazan pagos `PENDIENTE_REVISION`; un pago `APROBADO` o `REEMBOLSADO` responde `409`.
- Repetir el rechazo devuelve `200` con `idempotent: true` sin modificar nada.
- Si la reserva estaba `EN_REVISION`, vuelve a `PENDIENTE_PAGO` con un nuevo `expires_at` de 20 minutos para que el jugador reenvíe el comprobante.
- Un pago `RECHAZADO` ya no puede aprobarse (`409`).

3.5. POST /api/pagos/comprobante (foto del voucher)

Flujo de la app: el jugador elige la foto » `POST /api/pagos/comprobante` » recibe `comprobante_url` » la envía en `POST /api/reservas/:id/pagos`.

Cuerpo de Solicitud:

```
{ "reserva_id": 12, "mime_type": "image/jpeg", "contenido_base64": "/9j/4AAQSkZJRg..." }
```

Respuesta 201 Created:

```
{
  "ok": true,
  "data": {
    "bucket": "comprobantes",
    "path": "reserva-12/voucher-1730650000000.jpg",
    "comprobante_url": "supabase://comprobantes/reserva-12/voucher-1730650000000.jpg",
    "mime_type": "image/jpeg",
    "size_bytes": 184233,
    "almacenado": true
  }
}
```

Reglas:

- El bucket es **privado**. El ADMIN ve la imagen en `GET /api/pagos` mediante `comprobante_url_firmada`, un enlace temporal de 1 hora.
- Solo se aceptan JPEG, PNG o WEBP de hasta 5 MB. El cuerpo JSON de esta ruta admite hasta 8 MB, y el resto de la API mantiene 100 KB (`413` si se supera).
- Una reserva `CANCELADA` o `VENCIDA` no acepta comprobantes (`409`).
- El `comprobante_url` enviado al registrar el pago debe pertenecer a la misma reserva (`400` en caso contrario).
- Con base de datos real y sin `SUPABASE_SERVICE_ROLE_KEY` responde `502 STORAGE_ERROR`. En pruebas y modo local se valida sin guardar (`almacenado: false`).

3.6. POST /api/reservas/:id/cancelar

Cancela una reserva aplicando la regla de las 24 horas y revocando el QR.